

WebNet0

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

We found this packet capture and key. Recover the flag.

Hints

- Try using a tool like Wireshark.
 - How can you decrypt the TLS stream?
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - **Wireshark** – Packet analysis and TLS decryption
-

Complete Solution

Step 1: Download and Prepare

Download the files from the challenge page:

- `capture.pcap` – Network packet capture (encrypted TLS traffic)
 - `picopico.key` – RSA private key for decrypting TLS
-

Step 2: Basic File Inspection

First, verify the file types:

```
file capture.pcap picopico.key
```

Expected output:

```
capture.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4
(Ethernet, capture length 65535)
picopico.key: OpenSSH private key (no password)
```

The `capture.pcap` is a standard **PCAP** file containing network traffic.

The `picopico.key` is an **OpenSSH private key** (RSA key, no password).

Step 3: Search for Visible Text

Check if the flag appears as plain text within any file:

```
strings capture.pcap | grep -i "picoCTF"
```

Output: (no results)

The traffic is encrypted – the flag isn't directly visible.

Step 4: Inspect Metadata

```
exiftool capture.pcap picopico.key
```

Output:

```
===== capture.pcap
ExifTool Version Number      : 13.50
File Name                    : capture.pcap
...
File Type                    : PCAP
Link Type                    : BSD Loopback
Time Stamp                   : 2019:08:23 17:56:36.391755+02:00
```

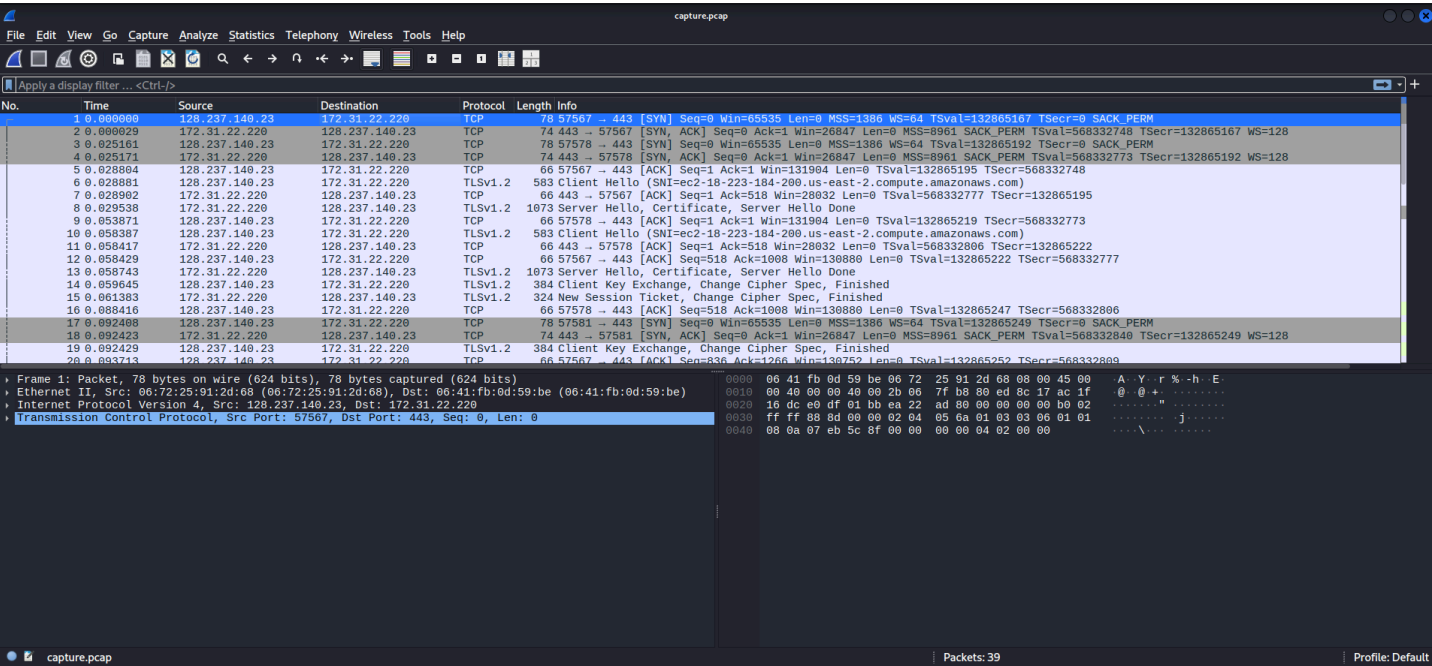
```
===== picopico.key
ExifTool Version Number      : 13.50
File Name                    : picopico.key
...
File Type                    : TXT
Line Count                   : 28
Word Count                   : 32
```

Nothing suspicious in the metadata.

Step 5: Open the PCAP in Wireshark

Launch Wireshark and open `capture.pcap` :

`wireshark capture.pcap`



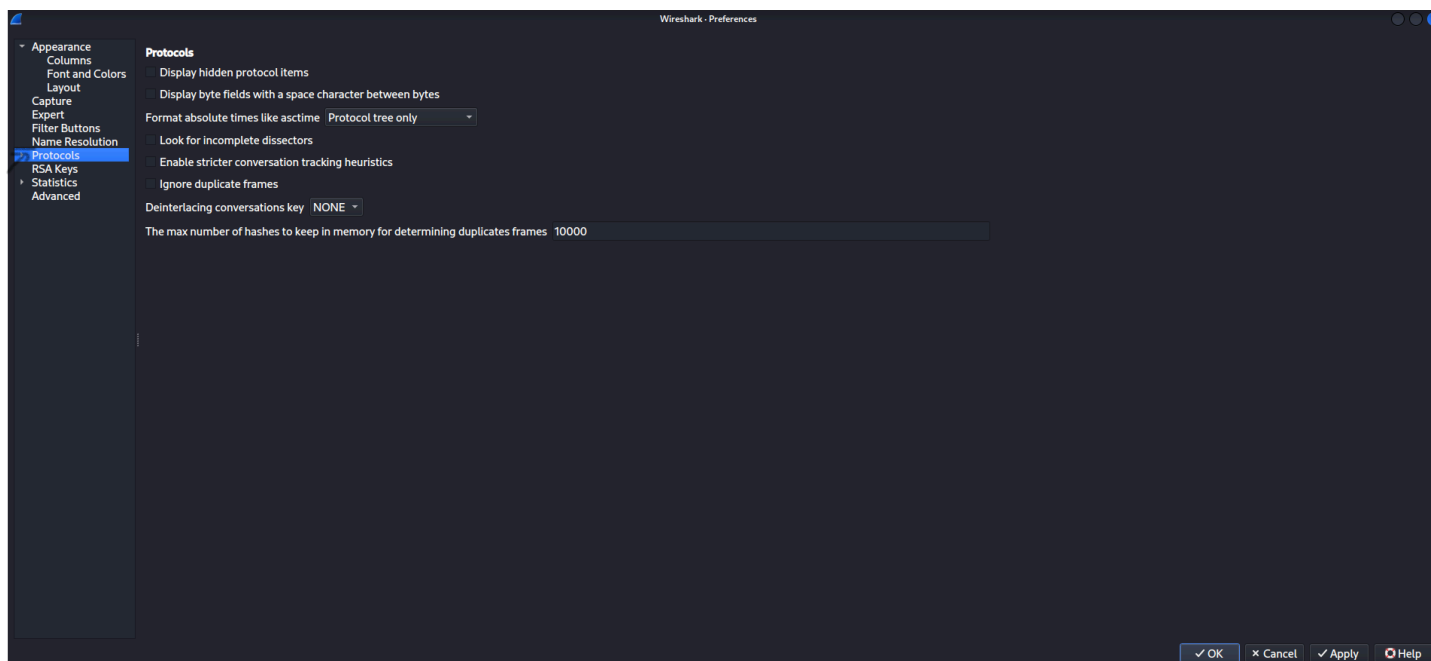
Initially, all traffic appears as “**Application Data**” – this is because the TLS (SSL) stream is encrypted. We need to decrypt it using the provided RSA private key.

Step 6: Configure TLS Decryption in Wireshark

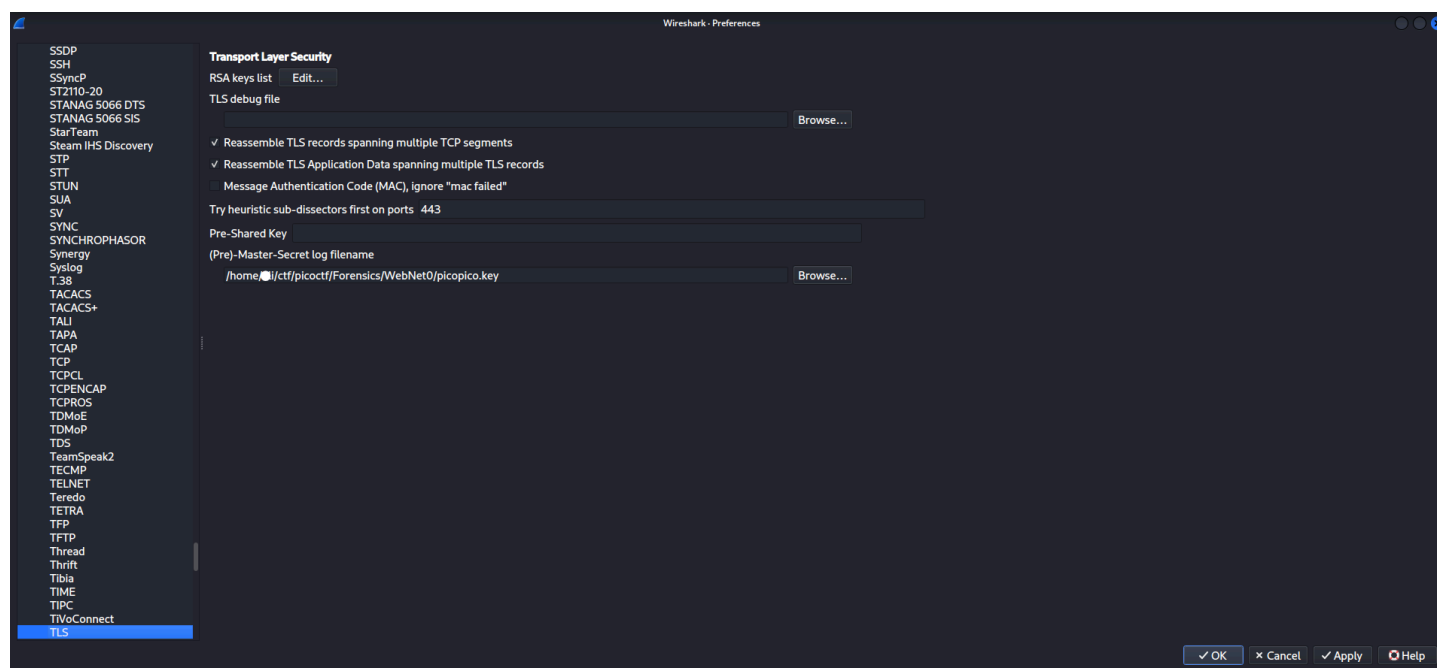
Wireshark can decrypt TLS traffic if you provide the server’s private key. The key (`picopico.key`) appears to be an RSA private key.

Step-by-step:

1. Go to **Edit** → **Preferences** (or press `Ctrl+Shift+P`)



2. Expand **Protocols** and scroll down to **TLS** (or **SSL** in older Wireshark versions)
3. In the “**(Pre)-Master-Secret log filename**” section, click the **Browse** button



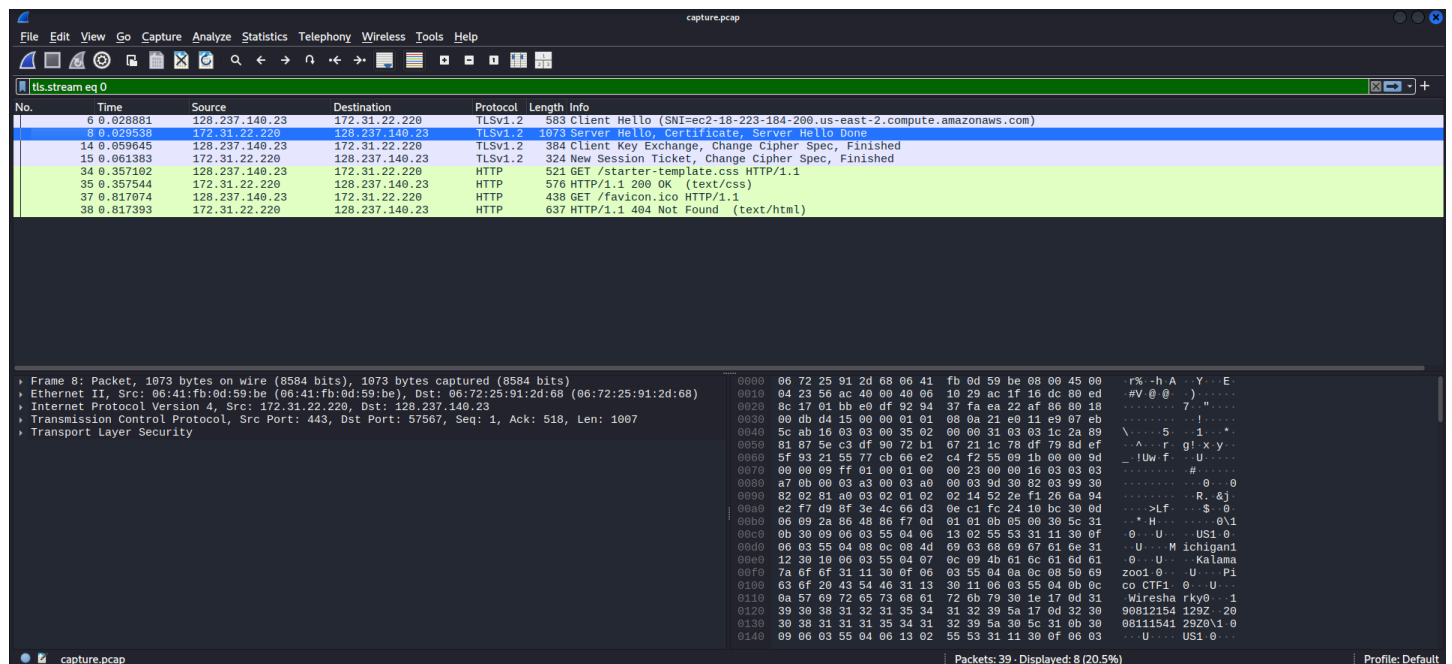
4. Add a new entry with the following:
 - **IP Address:** leave blank (applies to all IPs)
 - **Port:** 443 (default HTTPS port)
 - **Protocol:** http
 - **Key File:** browse and select `picopico.key` (or type the full path)

Note: If the key is in OpenSSH format (as shown by `file`), Wireshark may require it to be converted to PEM format. However, Wireshark typically handles OpenSSH keys directly.

5. Click **OK** to save

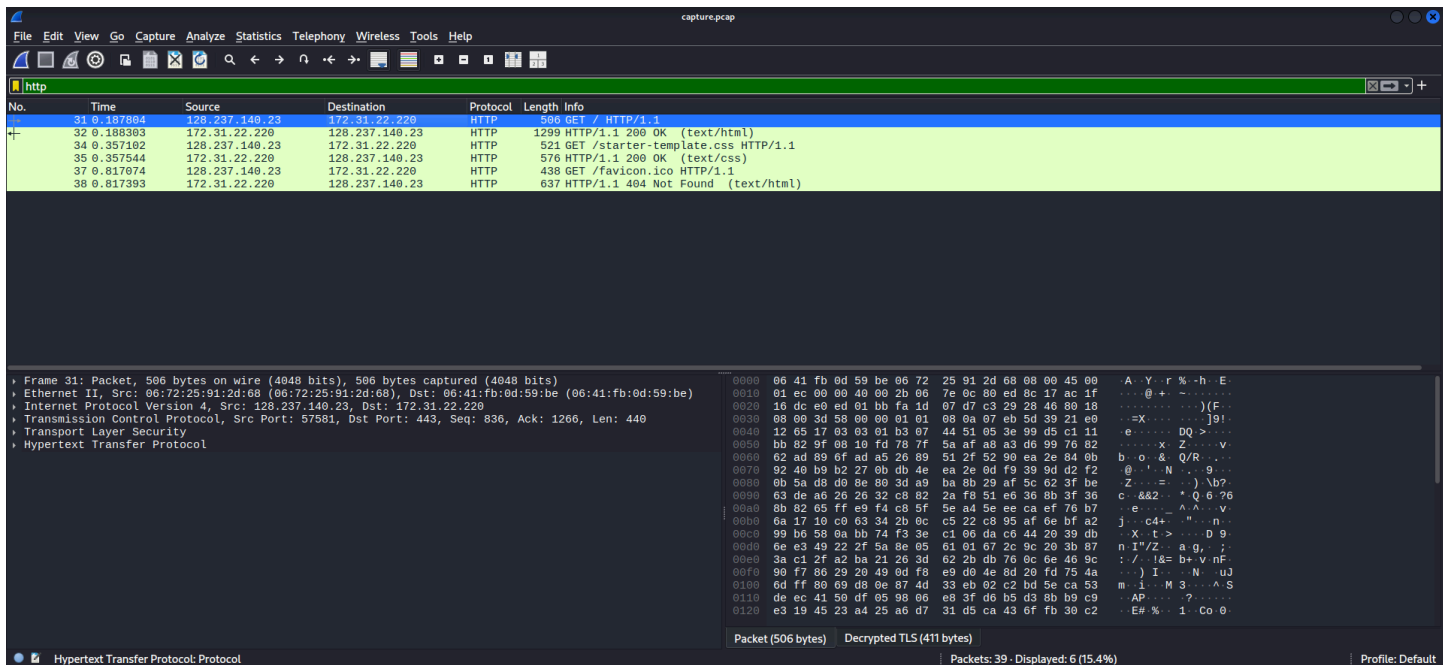
Step 7: Verify Decryption

After configuring the key, Wireshark will automatically re-analyze the packets. The encrypted “Application Data” will now be displayed as **HTTP** traffic.



Filter for HTTP traffic:

http



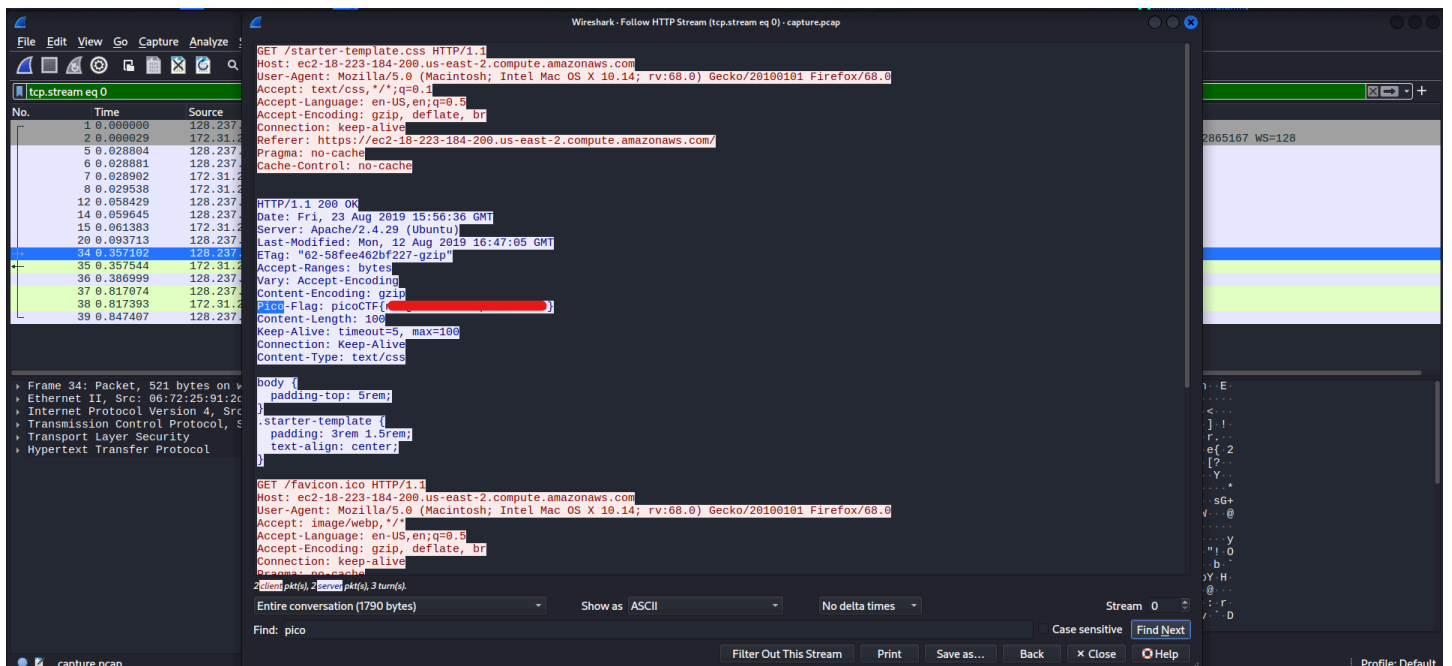
Step 8: Locate the Flag

Scroll through the HTTP responses or use the **Follow HTTP Stream** feature:

1. Right-click on any HTTP packet
2. Select **Follow** → **HTTP Stream**

Look for the **Pico-Flag** header in the response:

Pico-Flag: picoCTF{<REDACTED>}



Step 9: Alternative Method – Using tshark (Command Line)

If you prefer the command line, you can use `tshark` with the RSA key:

```
tshark -r capture.pcap -ouat:rsa_keys:"\"picopico.key\", \"\" -Y http -T fields -e http.response.line | grep -i "Pico-Flag"
```

✅ Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, after decrypting TLS with the provided key, the `Pico-Flag` HTTP header will reveal the complete flag.

🧠 Key Concepts

Concept	Description
TLS (Transport Layer Security)	Cryptographic protocol that encrypts HTTPS traffic.
RSA Private Key	Used to decrypt TLS sessions if the same key was used by the server.
Wireshark TLS Decryption	Feature that decrypts encrypted traffic when provided with the server’s private key.
SSL/TLS Handshake	The initial exchange where keys are negotiated; contains information needed for decryption.
Pico-Flag Header	Custom HTTP header used to transmit the flag in this challenge.

Pro Tips

1. **RSA keys work if the cipher suite uses RSA** – Modern TLS often uses Diffie-Hellman (Perfect Forward Secrecy), which cannot be decrypted with a static RSA key. This challenge likely uses an older RSA key exchange.
 2. **The key file must match the server** – Ensure the key corresponds to the certificate used by the server in the PCAP.
 3. **Check the TLS handshake** – Look for [Client Hello](#) and [Server Hello](#) packets. If you see [Server Key Exchange](#) with Diffie-Hellman parameters, the key won't decrypt it.
 4. **Use [Follow HTTP Stream](#)** – This reassembles the entire conversation, making it easier to find custom headers.
 5. **OpenSSH keys are supported** – Wireshark can load OpenSSH private key format directly.
 6. **Alternative tshark command** – The `-o` option overrides preferences, allowing you to specify the key file on the command line.
-

Alternative TLS Decryption Tools

Tool	Purpose
Wireshark (GUI)	Easiest for visual inspection
tshark (CLI)	Scriptable, good for automation
ssldump	Command-line SSL/TLS decoder
mitmproxy	Can decrypt TLS with key log files

Conclusion

The PCAP file `capture.pcap` contained encrypted TLS/HTTPS traffic. Using the provided RSA private key `picopico.key`, we configured Wireshark to decrypt the TLS stream. After decryption, the HTTP traffic became readable, and the flag was found in a custom HTTP header named `Pico-Flag`.

This challenge demonstrates:

- **TLS decryption** – Using a private key to decrypt captured encrypted traffic
- **Wireshark configuration** – Setting up RSA keys for protocol decryption
- **Network forensics** – Analyzing HTTPS traffic after decryption

Key takeaway: When you have a private key and a packet capture, you can often decrypt the TLS traffic in Wireshark. Look for the key exchange method (RSA vs. Diffie-Hellman) – RSA key exchanges allow decryption with the server's private key. Always check custom HTTP headers for flags.